Interpretable Rule Discovery for Anomaly-Selected Events

A practical introduction to ERL and our trigger-study workflow

Implementation snapshot: 11 October 2026

What problem does ERL solve?

An anomaly detector can identify unusual events without explaining what makes them unusual. An **existential rule list (ERL)** provides a complementary, human-readable description: it searches for short conditions that distinguish an anomaly-selected sample from a reference sample.

In our study, the reference represents ordinary minimum-bias collisions. The anomalous sample is a mixture: it can contain many ordinary events and a smaller contribution from another process. ERL compares these two samples; it does not need to know the mixture's components in advance.

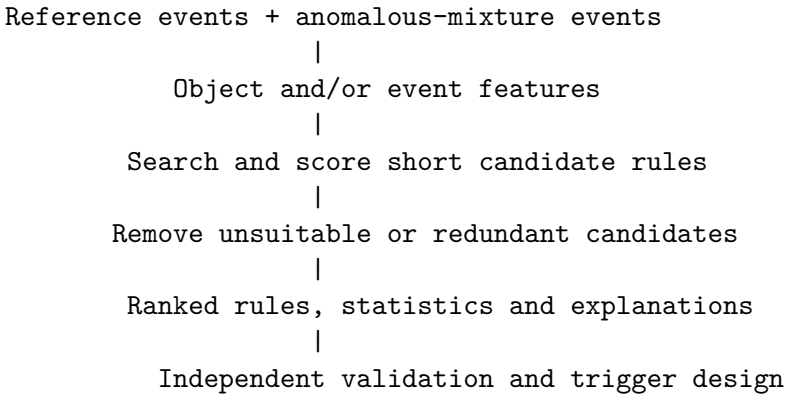
A discovered condition might read:

> Accept the event if at least two photons have transverse momentum above 20 GeV.

Transverse momentum, written p_T, is momentum perpendicular to the beam. In this report, a *cut* means a condition such as $p_T > 20$ GeV.

Such a condition is a candidate explanation and potentially a starting point for a trigger path. A **trigger** is a fast selection deciding which collision events to retain. ERL discovers conditions; it does not automatically establish their physical origin or their suitability for detector hardware.

The workflow in one view



The central idea is to turn a complex difference between datasets into a small number of inspectable selections. The output is a ranked collection of potentially overlapping rules, **not a sequence of exclusive decisions**. Conditions within a rule use AND (all must hold). Whole rules can be combined with OR (any rule may accept the event).

This report first explains the reusable model, then describes our external data preparation and scoring choices. It is an introduction to the current implementation, not a claim of validated CMS trigger performance.

What information can the model use?

Two complementary descriptions of each event

Object information describes individual reconstructed particles or jets: their type, transverse momentum, direction, isolation, and other properties. An event can contain a different number of objects from another event. Isolation measures nearby activity; a more isolated object has less of it.

Event information summarizes the whole collision: object counts, momentum sums, leading-object momenta, missing transverse momentum, and angular relationships. These summaries make multi-object conditions easy to express. *Leading* and *subleading* mean highest and second-highest transverse momentum within an object type. MET is the magnitude of the reconstructed transverse momentum imbalance, which may arise from invisible particles or measurement errors.

Input mode	What ERL can ask
Objects only	Does an object, or optionally an object pair, satisfy these properties?
Events only	Does this event summary satisfy these conditions?
Both	Do object and event requirements hold together?

Technically, an input tensor is a numeric array. Object inputs have shape **events** $\times$ **slots** $\times$ **properties**; event inputs have shape **events** $\times$ **variables**. A slot stores one object, or one pair when the separate pair tensor is selected. Unused slots are padded; a presence mask excludes them. Our runner constructs this mask from positive transverse momentum for single objects and reads the saved pair-presence mask for pairs. Event-only ERL works without a dummy object tensor.

Each feature also has a schema: its name, kind, and, where provided, units and validity information. This distinguishes numeric thresholds from categories such as photon versus jet. Both datasets must use compatible feature definitions.

How to read an object rule

```
There exists an object with:
    type = photon AND pt > 30 GeV AND isolation < 0.1
```

All three conditions must hold on **the same object**. They cannot be satisfied by a photon, an unrelated energetic jet, and a third isolated object. The word *existential* means that at least one matching object must exist. The core can instead require at least `k` objects matching the entire condition; our flexible runner currently uses one.

For a photon-plus-muon requirement, event counts can require at least one of each. Alternatively, pair inputs can require a photon member and a muon member with conditions on their individual and shared properties. Asking a single object to be both types is impossible. Leading and subleading event features can also express different thresholds for the first and second objects.

All pair conditions refer to **the same pair**, which contains two distinct objects. Different pair slots may share a member: two matching pairs do not necessarily mean four distinct particles.

Undefined is not the same as zero

An angle between two photons is undefined when only one photon exists. A linked validity flag ensures such an event cannot pass a cut on that angle, even if its stored placeholder is zero. These guards are automatic when declared in the schema; they do not add another search step. Linked flags are not independently searched by default. Meaningful absence remains expressible through counts.

How does ERL discover a rule?

Growing several promising ideas at once

ERL builds rules one condition at a time. At the first level it tries simple cuts, scores them, and keeps several promising alternatives. At the next level, it extends those alternatives with another condition joined by **AND**. This is called **beam search**. A partial rule is also called a *prefix*.

For example, the search might explore:

```
Level 1:  photon count above 20 GeV >= 1
Level 2:  above AND jet momentum sum > 50 GeV
Level 3:  above AND missing transverse momentum < 40 GeV
```

These are illustrative conditions, not results or recommended thresholds. Candidates retained from earlier levels can also appear in the final pool: the model is allowed to return a useful one-condition rule.

Numeric thresholds come from the data, with upper and lower cuts considered; category cuts test individual categories. Each extension is scored against the original input samples. Combined mode can add object or event conditions.

The main search controls

Control	Plain-language meaning
Maximum depth	Explicit conditions allowed per rule
Beam width	Prefixes kept for the next level
Threshold budget	Numeric thresholds per feature/prefix
Candidate cap	Rules saved in the candidate pool
Minimum mixture acceptance	Smallest accepted mixture fraction

A wider beam can preserve an initially modest condition that becomes useful after another cut. It costs more computation and is still not an exhaustive search. Increasing the final candidate cap cannot recover a path already lost at an earlier level. With many features, intermediate competition matters.

Optional object-feature priority

A priority such as `type_id, pt` requires a type cut, then a momentum cut, before other object features become eligible. Event cuts remain free and do not advance this object-cut sequence.

Entries can repeat and can specify a numerical comparison direction. For example, `type_id, pt:>, pt:<=` asks for type, then a lower momentum bound, then an upper momentum bound. Thresholds are learned from data, with each bound taking a separate search step.

Short rules remain eligible by default. An optional completion requirement retains only rules making **every listed cut**, in order and with any specified directions. Incomplete prefixes are still searched and reported; if none reaches completion, no rule is retained. Neither option guarantees the best rule or retention of every particle type.

The search also tracks surviving objects: equal event selections may leave different objects available for the next cut. This state is stored as compact bits and unused selections are discarded to reduce memory. Large pair inputs and wide searches can still require substantial RAM. `--progress` displays search counters and elapsed time, not a guaranteed completion time.

What makes a rule score well?

The core accepts a user-supplied score: a larger value means a more desirable rule. **Gini scoring is our study choice, not a fixed requirement of ERL.**

Gini: reward separation between the supplied samples

A rule divides the original data into a pass group and a fail group. Gini gain measures how much this division reduces mixing of the two training classes, accounting for both groups and their sizes.

In a group with anomalous-input fraction p, Gini impurity is $2p(1-p)$. It is largest when the two input classes are equally mixed and zero when a group contains only one class. Gain is the original impurity minus the size-weighted impurity after the division.

Consequently, a small but strongly enriched pass group can be valuable even when most anomalous-input events remain indistinguishable from the reference. However, pure Gini can also favor a reference-enriched pass group: it measures separation, not a preferred direction.

Penalize reference acceptance when designing triggers

Our flexible runner uses:

$$\text{score} = \text{Gini gain} - \lambda \times \text{reference acceptance}.$$

Acceptance means the fraction of that dataset passing the rule. Increasing λ makes accepting ordinary reference events more expensive. The penalty acts during search, not only after discovery. It is a preference, not a hard rate constraint or a universally optimal setting.

A worked example

Suppose each input contains 1,000 events. The following hypothetical rules are evaluated with unit event weights and $\lambda = 1$:

Quantity	Rule A	Rule B
Mixture events accepted	100	60
Reference events accepted	10	1
Mixture acceptance	10%	6%
Reference acceptance	1%	0.1%
Gini gain	0.01948	0.01472
Penalized score	0.00948	0.01372

Pure Gini prefers A. The penalized score prefers B: its reference rejection outweighs its reduced mixture coverage. Very large penalties can also favor fragile rare selections, so validation remains essential.

The score uses **mixture versus reference**, not true process labels. Mixture acceptance is not the efficiency for the unknown anomalous component. Unequal input sizes or event weights also affect Gini; the runner does not automatically rebalance the two classes. Each whole rule is scored against its complement, not just by the incremental improvement of its last condition.

How are redundant or unsuitable rules removed?

Pruning means removing candidates according to explicit criteria. The current model implements the following filters; the runner chooses which to enable.

Similarity: do these rules select almost the same events?

Similarity compares overlap between accepted selections. Its measure is **Jaccard overlap**: intersection divided by union. Identical selections have overlap one. A candidate is redundant only when the required comparisons pass for both datasets.

During beam search, surviving object slots and object-priority progress also matter because they affect future extensions. Final similarity compares accepted events. A higher similarity threshold makes pruning less aggressive. Beam and final similarity can be enabled independently; early filtering can remove a promising search path, whereas final filtering only edits the finished list. Similarity is about overlap, not physical equivalence: it may remove a useful refinement that accepts many of the same events. Raising its thresholds preserves more overlapping alternatives.

Logical pruning: does extra complexity earn its place?

Consider:

```
Simple:   photon pt > 30 GeV
Stricter: photon pt > 40 GeV AND isolation < 0.1
```

The stricter rule implies the simple one when the conditions refer to the same object and the object-count requirement is unchanged. It survives only if its score improves by **more than** the configured improvement threshold. This handles both tighter numeric bounds and additional AND conditions.

The improvement threshold can be positive, zero, or negative. Positive values require a meaningful score gain; zero requires any strictly positive gain. A negative value permits a limited score loss: `-0.01`, for example, lets a stricter rule survive this check when its loss is less than 0.01 against every applicable retained looser rule. This can preserve narrower descriptions, but does not disable similarity or rate filtering or change score-based ranking.

Equivalent rules are merged. Repeated bounds can be simplified: `pt > 50 AND pt > 40` is effectively `pt > 50`. Original steps remain available in the report.

This is a logical check, not an inference from matching training events. It does not automatically know relationships between different columns, such as leading photon momentum and photon counts above a threshold. It supports object, event, and combined rules, but may retain physically related alternatives.

Acceptance limit: does this rule fit the nominal rate budget?

The study translates a requested rate into a maximum reference acceptance:

$$\text{maximum acceptance} = \frac{\text{allowed output rate}}{\text{input rate}}.$$

For example, 40 kHz at 40 MHz means at most 0.1% reference acceptance. The runner converts the units; the core receives only that fraction.

Final selection first checks priority completion when requested, then acceptance eligibility, equivalence and logical pruning, and optional similarity last. An incomplete or over-budget simple rule cannot discard a rate-eligible extension. The limit applies **per rule**, not to the combined trigger menu, and currently uses training acceptance without an uncertainty margin. Adding OR paths requires a separate combined-rate evaluation.

The reusable model versus our study setup

The distinction matters when adapting this work to another experiment or another anomaly detector.

Reusable ERL functionality	Choices made around it
Read arrays and feature schemas	Convert raw Parquet into PT datasets
Search object/pair and/or event conditions	Choose particle collections and summaries
Enforce cut order, directions and optional completion	Choose a search structure and budgets
Evaluate a supplied scoring function	Supply reference-penalized Gini
Apply configured built-in pruning	Choose similarity and improvement settings
Apply a generic acceptance ceiling	Translate a trigger budget into that ceiling
Execute rules and their OR combinations	Produce logs, JSON reports and plots

The library does not read raw detector files, identify genuine signal processes, or implement a complete hardware trigger. Our scripts provide a particular research workflow around it.

What features are currently prepared?

The direct converter constructs inputs from electrons, tight muons, tight photons, and **PUPPI jets and MET**. PUPPI is a reconstruction approach intended to reduce contributions from additional simultaneous collisions, called pileup. PT files store the numeric tensors together with feature definitions and metadata.

The default object tensor stores up to 10 jets and four objects of each other type: **22 slots and 20 columns**. The columns include transverse momentum, direction, type, isolation, impact parameter, jet mass, constituent counts, charged fraction, energy ratio and charge, plus extra-property validity flags. Not every property applies to every object type.

An optional, separate pair tensor has **231 slots and 68 columns**. It contains the properties of both members, plus shared quantities such as momentum balance, invariant mass, angular separation and charge relationships, with validity flags. It is built from the same capped object collections. The runner selects pairs instead of single-object slots; event summaries can still be used alongside them. Pair construction and search are more expensive because the number of possible pairs grows approximately quadratically with the object count.

The default event tensor contains **193 columns, including validity flags**. Its main families are:

- Object counts, including counts above specified momentum thresholds.
- Per-type momentum sums, total object momentum sum, and MET.
- Leading and subleading momenta, momentum fractions and balances.
- Directions and angular separations, including object–MET angles.
- Charged particle-flow candidate counts and momentum sums in momentum and impact-parameter bins; leading/subleading lepton impact parameters.

Particle-flow candidates are lower-level reconstructed constituents. These charged-candidate summaries use the stored unweighted candidate momenta, not PUPPI-weighted momenta. Event summaries use full stored collections before object-slot truncation. Total object momentum sums do not remove overlaps between collections and are not producer ScalarHT.

These are converter defaults, **not fixed dimensions required by ERL**. Regenerating data is necessary to obtain newly added columns. The runner uses the schemas saved in each file; changing conversion code does not update old tensors. Some source fields, especially displacement-related ones, have production-specific limitations documented in the detailed report.

Running the study and reading its output

The current flexible script is `casual_tests/run_ERL_Gini_nested.py`. The following example assumes compatible reference and mixture PT files already exist. It illustrates an event-only run; it is not a universal recommended tune.

```
python casual_tests/run_ERL_Gini_nested.py \
  --signal data/example_mixture/anomalous.pt \
  --background data/example_mixture/reference.pt \
  --event-only \
  --reference-penalty 1 \
  --stricter-score-improvement 0.001 \
  --max-trigger-rate-khz 40 --input-rate-mhz 40 \
  --report reports/ERL_intro_example.txt
```

Remove `--event-only` to use both tensors; replace it with `--object-only` to use only objects. With objects enabled, `--basic-object-features` restricts them to momentum, direction and type. Add `--pair-tensor` to select saved pair inputs instead of single-object inputs. `--object-cut-priority type_id pt` enables ordered object cuts; pair inputs use member names such as `type_id_1` and `type_id_2`. Add `--require-complete-priority` to require the entire list in retained rules. Quote directed entries, such as `'pt:>'`, to avoid shell redirection. `--progress` displays search activity on the terminal. Beam and final similarity require their explicit enable flags; they are off in this example. Existing reports require `--overwrite-report` before replacement.

Know the runner defaults

The flexible runner searches up to four explicit cuts, carries a beam of 40, retains up to 300 candidates, and tries at most 32 numeric thresholds per feature and prefix. It requires at least 1% mixture acceptance. Both tensors are enabled by default, reference penalty is one, and logical pruning requires a strictly positive improvement by default. Priority, its completion requirement, progress, and a rate ceiling are off by default. Single-object inputs are used unless pairs are requested.

Search depth, beam width and candidate cap are constructor settings in this script, not current CLI options. The core has configurable parameters and different defaults. Older runners also have different settings.

Which quantities should a reader inspect?

The saved `.txt` report contains **JSON**, a structured machine-readable format. It records configuration, feature definitions, candidate conditions, scores, accepted counts, acceptances, step histories, and final-pruning decisions.

Read a rule alongside its mixture and reference acceptance, absolute counts, complexity, and pruning status. A high score alone is not a physics explanation. Global rank orders the full capped candidate pool by score; retained rank orders the surviving subset. Neither is a ranking by true signal purity. The report preserves ordered cut histories, priority progress and completion status. Incomplete rules rejected by the completion option remain visible with the pruning reason `incomplete_object_priority`. Candidates lost during beam search are not part of this saved pool.

Process labels are used only for diagnostics. A reported fraction of accepted events belonging to the anomalous input must not be confused with the fraction belonging to a genuine signal process. The acceptance plot places reference acceptance horizontally and mixture acceptance vertically, with rule definitions alongside. Low reference acceptance and high mixture acceptance are desirable for anomaly-enriched paths, subject to rate, stability, and practical constraints.

What conclusions are justified?

ERL can answer: **which short conditions distinguish these supplied datasets?** It cannot, by that fact alone, establish why an upstream anomaly detector selected the events or whether a discovered condition will remain useful in real data.

Important interpretation checks

A mixture is not pure signal. A rule can describe the background component, the upstream selection, or a detector response difference. In our demonstration, the mixture is assembled explicitly rather than selected by an anomaly detector.

Guards can imply object presence. A valid photon–MET angle already requires a photon. Its apparent usefulness may partly reflect photon presence rather than a new angular correlation. Extremely small positive MET can also have a physically unstable direction despite passing the finite-value guard.

Padding protection is explicit, not universal. Linked measurement guards exclude invalid placeholders. The current object eta/phi columns have no dedicated validity flags; missing values filled with zero remain an interpretation limitation; pair coordinates have their own guards. Producer-availability counts can also become reconstruction proxies.

Zero observed reference events does not mean zero rate. With 5,000 unweighted reference events, one accepted event corresponds to 0.02% acceptance, or 8 kHz at 40 MHz. A 40 kHz budget therefore corresponds to only five observed events: sampling uncertainty is important.

A clean explanation need not maximize the score. Correlated features can produce several descriptions of the same excess. Logical pruning removes some redundancy but does not understand all physical relationships between variables. The approximate search does not guarantee a familiar signature appears first.

Before treating a rule as a trigger proposal

1. Discover rules on one sample and measure them on independent held-out events.
2. Check stability across resampling, mixture fractions and reconstruction or pileup conditions; assess reference-rate uncertainty.
3. Examine what each condition physically selects, including validity effects.
4. Measure the OR of selected paths; per-path budgets do not constrain menu rate.
5. Re-evaluate practical threshold choices and hardware constraints such as precision, latency and resource use.

The current runner does not automatically split training from validation. Its printed acceptances describe the discovery inputs.

Further reading and scope

The companion `ERL_implementation.md` and `ERL_implementation.pdf` provide the full API, feature definitions, commands and implementation limitations. Core behavior is implemented under `src/set_based_models/rule_list/`; conversion is under `raw_event_conversion/`; study scoring and reporting are under `casual_tests/`.

The worked scoring example is hypothetical. Search settings and rate estimates should be validated for the intended datasets and use case.